PHASE 2 · BUILDING · V0.1 FOR TEAM REVIEW

RoleReady build architecture

A TOGAF-style view of what we need to build the must-have flow, who builds which part, and the contracts that let four people work separately and still fit together on Saturday.

Scope must-haves only (voice and email report come later)

Team Group 44 · A (Pro, frontend lead), B, C, D · all in the UK

Deadline Sun 1 Nov 10:00 IST = 04:30 UK. We aim to submit Sat 31 Oct 22:00 UK **All times** UK (GMT)

Architecture on one page

Four TOGAF architecture domains, top to bottom. Each box shows the person who owns it. Colours follow the ArchiMate convention (yellow business, blue application, green technology) with data in purple.

A Pro · frontend B data, auth, platform C question engine D feedback engine, deploy, QA

Business

ADM phase B

ACTOR: JOB SEEKER WITH AN INTERVIEW IN 1–14 DAYS

Sign in → Upload resume PDF + paste JD → Get 6 grounded questions → Answer in text → Score, one fix, stronger answer → Reopen from History

DEMO SUCCESS: FIRST SCORED FEEDBACK IN UNDER 3 MINUTES, 6/6 QUESTIONS TIED TO A RESUME LINE OR JD REQUIREMENT

Data

ADM phase C

User B

Supabase `auth.users`, email + password

Session B

JD text, resume lines R1…Rn, JD requirements J1…Jn, status, overall score

Question ×6 B

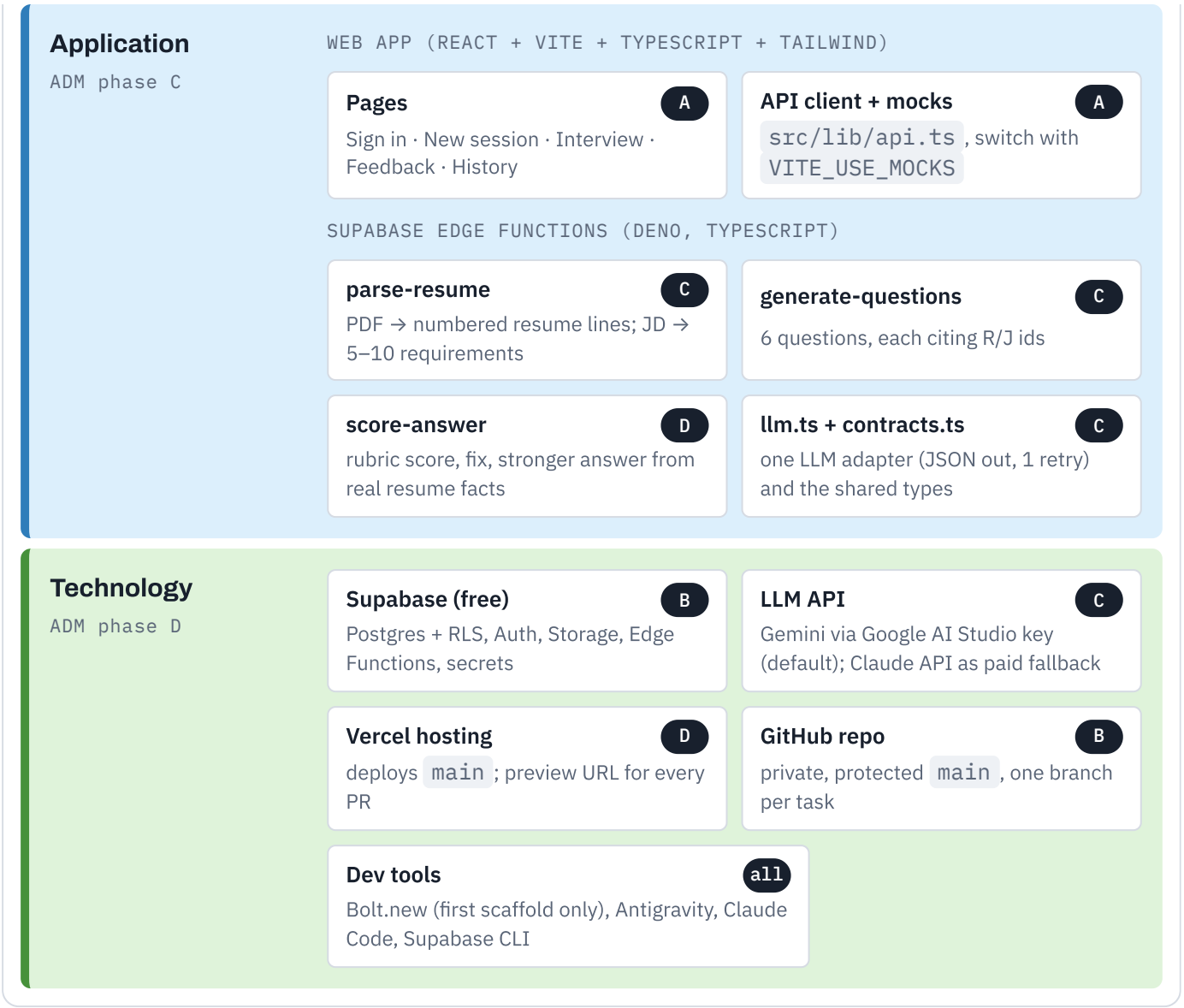
text, type, why we're asking, cited R/J ids

Answer + feedback B

4 scores (1–5), overall 0–100, what worked, one fix, stronger answer, unsupported claims

Resume PDF B

private bucket `resumes/{user_id}/{session_id}.pdf`



Decisions this rests on

Your AI subscriptions cover the coding, not the app

Claude and Google AI subscriptions pay for the tools you build with (Claude Code, Antigravity, Gemini). The app's own LLM calls need an API key, which those subscriptions don't include. Use a free Gemini API key from Google AI Studio for the app; it is rate-limited but enough for a demo. Keep the Claude API as an optional paid fallback behind `LLM_PROVIDER`. Free-tier prompts may be used by Google to improve its products, so demo with sample resumes, not real ones.

No custom backend server

Supabase does auth, data, files and the three AI functions. Fewer moving parts to deploy in 3 days.

LLM keys never reach the browser

Only Edge Functions call the LLM, using a key set with `supabase secrets set`.

Functions run as the user

They use the caller's token, so row-level security protects every read and write. No service-role key in code.

Grounding by id

Resume lines become R1…Rn and JD requirements J1…Jn. Questions and feedback cite ids, which is how the UI shows "why we're asking".

Email + password sign-in

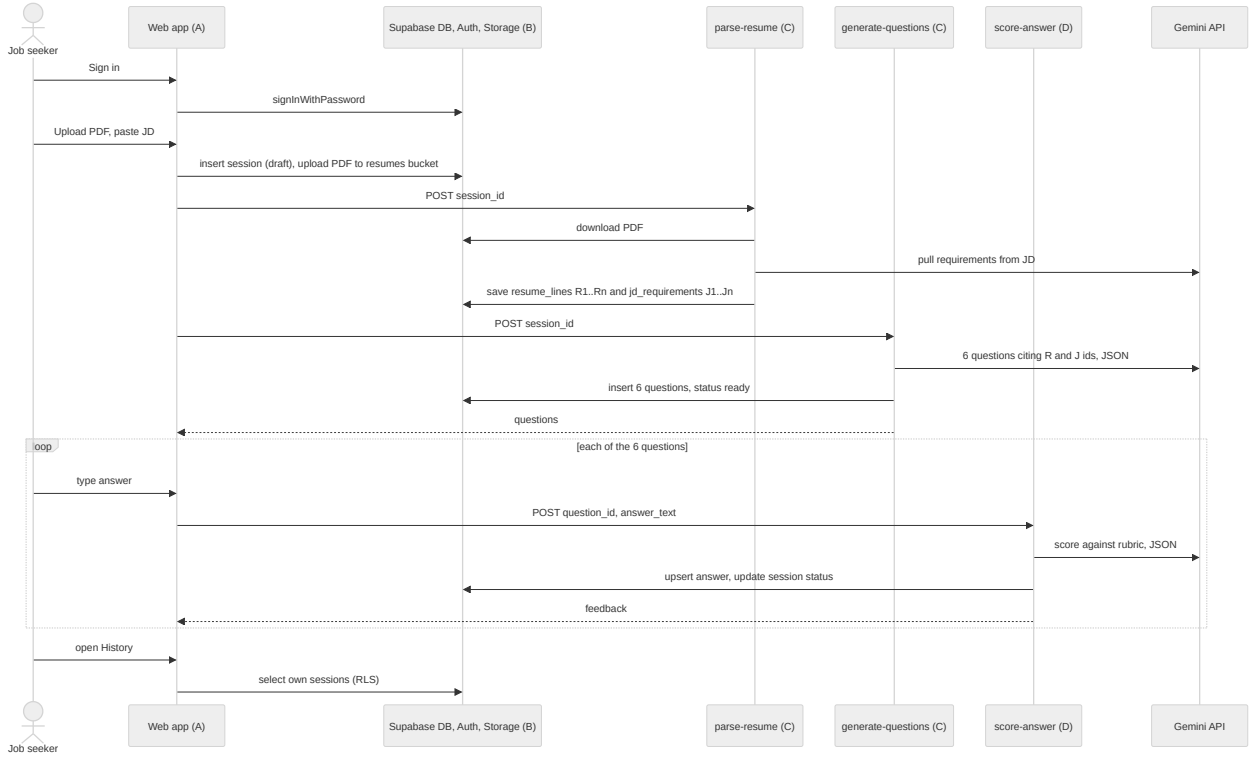
Turn "confirm email" off for the hackathon. Magic links hit Supabase's default email limit quickly during demos.

Mocks first

The UI runs on fixture data from hour one, so frontend work never waits for the backend.

How one session runs

Every arrow into a function is one of the three endpoint contracts below.



Who builds what

Each person owns separate folders, so most days nobody touches another person's files.
The contracts file is the only shared surface.

A Pro
Frontend and product lead

OWNS

`src/`, `contracts.ts` (keeper), the demo and submission

BUILDS

- Scaffold in Bolt.new, push to GitHub, then continue in Antigravity or Claude Code
- Pages: Sign in, New session (PDF + JD), Interview (6 questions with "why we're asking" tags), Feedback panel, History
- `api.ts` calling the 3 functions; `mocks/` built from D's fixtures

DONE WHEN

The whole flow clicks through on mocks, then on the real backend with mocks off.

B Person B
Data, auth and platform

OWNS

the GitHub repo, `supabase/migrations/`, the Supabase project, secrets

BUILDS

- Create the GitHub repo and the Supabase project, invite A, C and D
- Apply `0001_init.sql`: tables, RLS, private `resumes` bucket
- Auth settings, a demo user, `supabase gen types` into `src/types/db.ts`
- Set the LLM secret; review any schema change

DONE WHEN

A signed-in user can insert a session and upload a PDF, and cannot read anyone else's.

C Person C
Question engine

OWNS

`functions/_shared/llm.ts`, `parse-resume/`, `generate-questions/`

BUILDS

- `llm.ts` first (Fri morning): `callJson<T>(system, user)` for Gemini, Claude as fallback
- PDF text with `npm:unpdf`, split into R-lines; JD into J-requirements
- Question prompt: mix of behavioural, technical and role-fit; each cites ≥1 id

DONE WHEN

A curl call with each fixture returns 6 questions, all citing real ids.

D Person D
Feedback engine, deploy and QA

OWNS

`score-answer/`, `fixtures/`, the Vercel project

BUILDS

- Fixtures first: 3 sample resumes + JDs, a strong and a weak answer for each
- Rubric prompt (relevance, evidence, structure, impact), stronger answer that uses only resume facts
- Vercel deploy with env vars; end-to-end test checklist

DONE WHEN

Weak answers score clearly below strong ones and invented claims show up as unsupported.

Everyone's task list

Every step of the playbook (Phase 1 to Phase 3) has one owner. Step numbers match the workbook. Phase 1 is already drafted, so the Phase 1 tasks are review and validation.

A Pro · frontend, product, demo

STEP	TASK	DUE
P1	Walk the team through the Phase 1 doc and freeze the scope at kickoff	**Fri 09:30**
5.2	Screens and actions table (5 screens) for the PRD	**Fri 10:00**
6–7	Review the PRD and the Antigravity starting prompt, then share both	**Fri 10:30**
8	Generate the first UI from the starting prompt, push to the repo, add mocks	**Fri 12:00**
8	All 5 screens clickable on mocks, including "why we're asking" tags and the feedback panel	**Fri 20:00**
9.2	Delete a session from History, so every entity has full create, read, update and delete	**Sat 10:00**
8	Switch mocks off and run the real flow with C and D	**Sat 10:00**
12.1	Demo script and Loom narration (2–3 min)	**Sat 18:00**
Submit	Fill in the submission form in one sitting (no autosave): live link, Loom, deck	**Sat 22:00**

B data, auth, platform, automation

STEP	TASK	DUE
G1–G2	Create private repo `aiap-c15-team44-roleready`, add A (`psaranconsultancy`), C and D with write access, protect `main`	**Thu 21:00**
5.1	Create the Supabase project, invite A, C and D	**Thu 21:00**
9.1	Apply `0001_init.sql`, turn off email confirmation, create a demo user	**Fri 12:00**
9.1	Generate `src/types/db.ts` and `lib/supabase.ts`; set the Gemini secret	**Fri 14:00**
9.2	CRUD checklist for sessions, questions and answers; check one user can't see another's data	**Fri 20:00**
10.1	Automations table for the PRD (sign-up, session created, session completed)	**Sat 10:00**
Should	n8n + Gmail: email the session report when a session is completed	**Sat 17:00**
Deck	Architecture slide from this page	**Sat 18:00**

C question engine, voice

STEP	TASK	DUE
P1	Sense-check the MoSCoW table and the "resume-grounded questions" differentiator	**Fri 09:30**
5.1	`llm.ts` : `callJson<T>` for Gemini, Claude fallback, one retry	**Fri 11:00**
10	`parse-resume` : PDF to R-lines, JD to J-requirements	**Fri 15:00**
10.2	`generate-questions` : 6 questions, each citing real R/J ids (our "wow" feature)	**Fri 20:00**
11	Tune the question prompt on all 3 fixtures with D	**Sat 12:00**
Should	Retell voice mock on a branch; fallback is browser dictation in the answer box	**Sat 17:00**
Deck	"How it works" slide: grounding by resume line and JD requirement	**Sat 18:00**

D feedback engine, QA, deploy, deck

STEP	TASK	DUE
P1	Ask 2–3 job-seeking friends "how did you prep for your last interview?"; send quotes for the problem slide	**Fri 18:00**
11.1	Fixtures: 3 sample resumes + JDs, a strong and a weak answer each; write the test scenario	**Fri 11:00**
10	`score-answer` : rubric, one fix, stronger answer using only resume facts	**Fri 20:00**
8	Vercel project linked to `main` , env vars set, preview URL per PR	**Fri 20:00**
11.2	Run the scenario, keep the bug and gap log, assign fixes	**Sat 10:00–16:00**
11.3	Stability check: full flow works twice in a row on the live URL	**Sat 14:00**
Deck	Finish the deck from the template (problem and competitor slides are drafted from Phase 1)	**Sat 19:00**
12	Record and trim the Loom with A	**Sat 21:00**

Interfaces between people

INTERFACE	PRODUCER	USED BY	DEFINED IN	FROZEN BY
Tables, RLS, storage bucket	B	A, C, D	`supabase/migrations/0001_init.sql`	**Fri 12:00**
Shared TypeScript types	A (keeper)	everyone	`supabase/functions/_shared/contracts.ts`	**Fri 12:00**
`POST /parse-resume`	C	A	contracts.ts, endpoint 1	**Fri 12:00**

INTERFACE	PRODUCER	USED BY	DEFINED IN	FROZEN BY
`POST /generate-questions`	C	A	contracts.ts, endpoint 2	**Fri 12:00**
`POST /score-answer`	D	A	contracts.ts, endpoint 3	**Fri 12:00**
`callJson<T>(system, user)`	C	D	`supabase/functions/_shared/llm.ts`	**Fri 11:00**
Sample resumes, JDs, answers	D	A (mocks), C, D	`fixtures/`	**Fri 11:00**

▼ contracts.ts (types and the 3 endpoints)

supabase/functions/_shared/contracts.ts

```
// RoleReady shared contracts v0.1 (must-haves only)
// Lives at supabase/functions/_shared/contracts.ts so both the Deno edge
// functions and the React app (via a Vite alias "@contracts") import the same
file.
// Change it only through a PR approved by Person A plus the owner of the
affected endpoint.

// ---------- Grounding ids ----------
// Resume lines are numbered R1..Rn, JD requirements J1..Jn.
// Questions and feedback cite these ids so the UI can show "why we're asking".
export interface ResumeLine { id: string; text: string }      // { id: "R7",
text: "Led migration of billing to Stripe, cut failed payments 30%" }
export interface JdRequirement { id: string; text: string }   // { id: "J3",
text: "Experience with payment integrations" }

// ---------- Rows (mirror supabase/migrations/0001_init.sql) ----------
export type SessionStatus = "draft" | "ready" | "in_progress" | "completed";

export interface Session {
  id: string;
  user_id: string;
  role_title: string | null;
  company: string | null;
  jd_text: string;
  resume_path: string | null;          // "resumes/{user_id}/{session_id}.pdf"
in Storage
  resume_lines: ResumeLine[] | null;   // filled by parse-resume
  jd_requirements: JdRequirement[] | null;
  status: SessionStatus;
```

```
  overall_score: number | null;        // 0-100, average of answer.overall, set
when completed
  created_at: string;
  completed_at: string | null;
}

export type QuestionType = "behavioural" | "technical" | "role_fit" |
"situational";

export interface Question {
  id: string;
  session_id: string;
  position: number;                    // 1..6
  text: string;
  type: QuestionType;
  why_asking: string;                  // one sentence shown under the question
  resume_refs: string[];               // e.g. ["R7"]  (at least one of
resume_refs / jd_refs is non-empty)
  jd_refs: string[];                   // e.g. ["J3"]
}

export interface Scores {              // each 1..5
  relevance: number;                   // answers the question asked, for this
role
  evidence: number;                    // uses real facts from the resume
  structure: number;                   // clear STAR-style story
  impact: number;                      // concrete outcome or number
}

export interface Feedback {
  question_id: string;
  answer_text: string;
  scores: Scores;
  overall: number;                     // 0..100 = round((sum of scores - 4) /
16 * 100)
  what_worked: string;
  one_fix: string;
  stronger_answer: string;             // rewritten using only facts present in
resume_lines
  unsupported_claims: string[];        // claims in the answer not backed by the
resume
}

// ---------- Edge function endpoints ----------
// All: POST {SUPABASE_URL}/functions/v1/<name>
// Headers: Authorization: Bearer <user access token>, Content-Type:
application/json
// Errors: non-2xx with body ApiError.

export interface ApiError { error: { code: "bad_request" | "unauthorized" |
"not_found" | "llm_failed" | "parse_failed"; message: string } }

// 1. parse-resume  (owner: Person C)
```

```
// Reads the PDF at session.resume_path, extracts text, numbers the lines,
// pulls 5-10 requirements out of jd_text, saves both on the session.
export interface ParseResumeRequest { session_id: string }
export interface ParseResumeResponse { resume_lines: ResumeLine[];
jd_requirements: JdRequirement[] }

// 2. generate-questions  (owner: Person C)
// Creates exactly 6 questions, inserts them, sets session.status = "ready".
// Calling it again replaces the 6 questions (only while no answers exist).
export interface GenerateQuestionsRequest { session_id: string }
export interface GenerateQuestionsResponse { questions: Question[] }

// 3. score-answer  (owner: Person D)
// Scores one answer, upserts the answers row (one per question; re-answering
overwrites).
// Sets session.status = "in_progress", or "completed" + overall_score once all
6 are answered.
export interface ScoreAnswerRequest { question_id: string; answer_text: string }
// answer_text 20..4000 chars
export interface ScoreAnswerResponse extends Feedback {
  session_completed: boolean;
  session_overall: number | null;
}
```

▼ **0001_init.sql (tables, row-level security, storage)**

supabase/migrations/0001_init.sql

```
-- RoleReady schema v0.1 (owner: Person B)
-- Put at supabase/migrations/0001_init.sql and run with `supabase db push`
-- (or paste into the Supabase SQL editor).
-- Every table carries user_id so one simple RLS rule covers it.
-- Edge functions call the database with the user's own token, so RLS applies
there too.

create table public.sessions (
  id              uuid primary key default gen_random_uuid(),
  user_id         uuid not null default auth.uid() references auth.users(id) on
delete cascade,
  role_title      text,
  company         text,
  jd_text         text not null,
  resume_path     text,
  resume_text     text,
  resume_lines    jsonb,
  jd_requirements jsonb,
  status          text not null default 'draft'
                  check (status in ('draft', 'ready', 'in_progress',
'completed')),
  overall_score   smallint check (overall_score between 0 and 100),
```

```
  created_at      timestamptz not null default now(),
  completed_at    timestamptz
);

create table public.questions (
  id          uuid primary key default gen_random_uuid(),
  session_id  uuid not null references public.sessions(id) on delete cascade,
  user_id     uuid not null default auth.uid() references auth.users(id) on
delete cascade,
  position    smallint not null check (position between 1 and 6),
  text        text not null,
  type        text not null check (type in ('behavioural', 'technical',
'role_fit', 'situational')),
  why_asking  text not null,
  resume_refs text[] not null default '{}',
  jd_refs     text[] not null default '{}',
  unique (session_id, position)
);

create table public.answers (
  id                 uuid primary key default gen_random_uuid(),
  question_id        uuid not null unique references public.questions(id) on
delete cascade,
  session_id         uuid not null references public.sessions(id) on delete
cascade,
  user_id            uuid not null default auth.uid() references auth.users(id)
on delete cascade,
  answer_text        text not null,
  scores             jsonb not null,
  overall            smallint not null check (overall between 0 and 100),
  what_worked        text not null,
  one_fix            text not null,
  stronger_answer    text not null,
  unsupported_claims text[] not null default '{}',
  created_at         timestamptz not null default now(),
  updated_at         timestamptz not null default now()
);

create index on public.sessions (user_id, created_at desc);
create index on public.questions (session_id);
create index on public.answers (session_id);

alter table public.sessions  enable row level security;
alter table public.questions enable row level security;
alter table public.answers   enable row level security;

create policy "own sessions"  on public.sessions  for all to authenticated
  using (user_id = (select auth.uid())) with check (user_id = (select
auth.uid()));
create policy "own questions" on public.questions for all to authenticated
  using (user_id = (select auth.uid())) with check (user_id = (select
auth.uid()));
create policy "own answers"   on public.answers   for all to authenticated
```

```
  using (user_id = (select auth.uid())) with check (user_id = (select
auth.uid()));

-- Private bucket for resume PDFs, stored as {user_id}/{session_id}.pdf
insert into storage.buckets (id, name, public)
values ('resumes', 'resumes', false)
on conflict (id) do nothing;

create policy "resumes: read own"   on storage.objects for select to
authenticated
  using (bucket_id = 'resumes' and (storage.foldername(name))[1] = (select
auth.uid())::text);
create policy "resumes: upload own" on storage.objects for insert to
authenticated
  with check (bucket_id = 'resumes' and (storage.foldername(name))[1] = (select
auth.uid())::text);
create policy "resumes: replace own" on storage.objects for update to
authenticated
  using (bucket_id = 'resumes' and (storage.foldername(name))[1] = (select
auth.uid())::text);
```

Working together

Repo layout and owners

```
roleready/
├─ src/                          A  React
app
│  ├─ pages/                     A
SignIn, NewSession, Interview, History
│  ├─ components/                A
│  ├─ lib/api.ts                 A  calls
functions or mocks
│  ├─ lib/supabase.ts            B  client
setup
│  ├─ types/db.ts                B
generated types
│  └─ mocks/                     A  built
from fixtures
├─ supabase/
│  ├─ migrations/                B
│  └─ functions/
│     ├─ _shared/contracts.ts    A  change
by PR only
│     ├─ _shared/llm.ts          C
│     ├─ parse-resume/           C
│     ├─ generate-questions/     C
│     └─ score-answer/           D
├─ fixtures/                     D
├─ .github/CODEOWNERS            A
```

Repo and branches

1. B creates the private GitHub repo and adds A (`psaranconsultancy`), C and D with write access. A pushes the first UI scaffold into it.
2. Protect `main`: changes only by pull request, one approval, squash merge. Vercel deploys `main`.
3. Short branches named `owner/task`, for example `c/generate-questions`. Merge to `main` at least twice a day.
4. Stop editing in Bolt once the repo is shared. Bolt rewrites files and will cause conflicts; use Antigravity or Claude Code on your branch.
5. Put folder ownership and "never edit contracts.ts without a PR" in `CLAUDE.md` / `AGENTS.md` so everyone's coding agent follows it.
6. Schema changes only as new migration files, reviewed by B.

```
├─ CLAUDE.md / AGENTS.md      A  rules
for coding agents
└─ .env.example
```

Keys and environment

VALUE	WHERE IT LIVES	WHO HAS IT	SHARE HOW
`VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`	`.env.local` and Vercel env	everyone	Team chat is fine; RLS protects the data
`GEMINI_API_KEY` (cloud)	Supabase secrets	B sets it	Never in git, never with a `VITE_` prefix
`GEMINI_API_KEY` (local)	`supabase/functions/.env`, gitignored	C, D	Each person makes their own free key in AI Studio
Supabase `service_role` key	Supabase dashboard only	B	Not needed by the app; do not share
Supabase CLI access	`supabase login` + `supabase link`	all four	B invites each person to the Supabase org

Timeline to submission (UK time)

The deadline is Sun 1 Nov 10:00 IST, which is 04:30 on Sunday in the UK. Everything finishes on Saturday evening, leaving more than six hours of buffer.

WHEN (UK)	MILESTONE	EVERYONE HAS
Thu 29 Oct 21:00	Accounts ready	Repo and Supabase project created, everyone invited; each person has a free Gemini key
Fri 30 Oct 09:00	30-minute kickoff on this page	Phase 1 frozen, tasks confirmed, everyone linked to repo and Supabase
Fri 30 Oct 12:00	Contracts frozen	Schema applied, `contracts.ts` and `llm.ts` merged, fixtures in, UI scaffold on mocks
Fri 30 Oct 20:00	Each part works on its own	Full flow clickable on mocks; all 3 functions deployed and passing curl tests
Sat 31 Oct 10:00	Integration	Mocks off, full flow working on the Vercel preview URL
Sat 31 Oct 14:00	Must-haves done	Flow works twice in a row on the production URL with a demo account

WHEN (UK)	MILESTONE	EVERYONE HAS
Sat 31 Oct 17:00	Feature freeze	Should-haves (voice, email report) merged only if they work; otherwise left out
Sat 31 Oct 21:00	Demo assets done	Final deploy, Loom (2–3 min), deck
Sat 31 Oct 22:00	Submit	Form filled in one sitting (no autosave); 6½ hours of buffer before 04:30

Suggested check-ins: 15 minutes at 09:00, 13:00 and 19:00 UK on Friday and Saturday.
Starter files (contracts, schema, env template) are in the project folder under `phase2-architecture/starter/`.